

Workshop Construyendo Software con IA



Issue-Driven, BDD y Epic-Driven: qué hace cada una y cuándo elegirla.

Ronny Freitas

Founder de shipease.ai — transformación digital como RaaS

NUEVE AÑOS Plataformas con millones de transacciones diarias, infraestructura de pagos e infraestructura de datos para investigación en proteómica

HOY Consultoría de producto con AI y entrega de resultados como servicio, no de horas

ENCUÉNTRAME shipease.ai · github.com/ronnycoding

POR QUÉ ESTA CHARLA

Todo lo que van a ver salió de trabajo real.

Los comandos, los agentes y las plantillas del repo son la configuración que uso cada día. Está abierta: clónenla, cámbienla, rómpenla.

Agenda

El principio común detrás de las tres	4 min
---------------------------------------	-------

00 · Anatomía de la configuración	7 min
-----------------------------------	-------

01 · Issue-Driven Development + demo	7 min
--------------------------------------	-------

02 · Behavior-Driven Development + demo	10 min
---	--------

03 · Epic-Driven Development + demo	11 min
-------------------------------------	--------

Cómo elegir, cheatsheet y preguntas	6 min
-------------------------------------	-------

El límite del prompt suelto

Pedir una feature completa en una conversación funciona en lo pequeño. Al crecer aparecen siempre los mismos tres síntomas.

01

El contexto se llena

Una sola conversación carga el backend, el frontend, el esquema y los tests a la vez.

02

Todo pasa en serie

Trabajo que podría correr en paralelo espera su turno en la misma sesión.

03

Nadie revisa nada

Sin criterios de aceptación escritos, no hay contra qué validar el resultado.

Descomponer, aislar, especializar

Las tres metodologías comparten los mismos seis beneficios.

Contexto acotado Los sub-issues mantienen el uso de tokens manejable.	Trabajo paralelo Varios agentes trabajan de forma simultánea.	Dependencias claras Los grafos Mermaid muestran los puntos de integración.
Puertas de calidad Cada PR recibe una revisión especializada.	Progreso visible El issue padre refleja el avance completo.	Especialización El experto correcto para cada componente.

Las tres, en paralelo

	Individual	Behavioral	Scaled
FLUJO	/issue → /task	/user-story → /issue → /task	/work-on-opens
ENTRADA	Una feature	Un comportamiento de usuario	Un tablero de prioridades
SALIDA	1 PR	1 PR por sub-issue	N PRs en M epics
PARALELISMO	Sesión única	Sesión única	Multi-epic, multi-agente
COMPLEJIDAD	Baja	Media	Alta

00

Anatomía de la configuración

~/claude/ – comandos · skills · agentes · plantillas

Las cuatro piezas de ~/.claude

Todas son archivos markdown en disco. Cambian por dónde entran.

commands/*.md Comando Una plantilla de workflow multi-fase que tú disparas. Se invoca: /nombre	skills/<nombre>/ Skill Experiencia de dominio empaquetada en su propia carpeta. Automática o explícita	agents/*.md Agente Un subagente con contexto propio y su tier de modelo. Lo elige la tarea	templates/GH_*.md Plantilla El formato fijo de issues, sub-issues, PRs y user stories. Se carga, no se invoca
--	--	--	---

Un comando: `commands/issue.md`

```
---  
description: Create an epic + sub-issues  
argument-hint: <feature description>  
allowed-tools: Bash(gh:*), Read, Task  
---
```

Phase 1 – Repository research

Analiza el stack y las convenciones antes de escribir nada.

Phase 4 – Parent issue

Carga templates/GH_PARENT_ISSUE_TEMPLATE.md y rellena el alcance con `$ARGUMENTS`.

Rules

Puntos Fibonacci. Nunca tiempo.

El frontmatter es el contrato

Qué hace el comando, qué argumento espera y qué herramientas tiene permitido tocar.

El cuerpo son fases numeradas

/issue tiene siete: research del repo, skills, mejores prácticas, clasificación, issue padre, sub-issues y QA.

`$ARGUMENTS` es lo que escribes

Lo que va después de /issue entra en la plantilla por esa variable.

El nombre del archivo es el comando

Añade `mi-flujo.md` a la carpeta y ya tienes `/mi-flujo`.

Una skill: skills/commit-message/

SKILL.md

name: commit-message

description: Use when writing a
commit message.

Instructions

Lee el diff. Escribe el mensaje en
Conventional Commits. Una línea.

La carpeta puede traer más

reference.md · examples/ · scripts/

No la invocas tú

Claude la carga cuando la conversación encaja con su descripción.

La descripción es el disparador

Empieza por «Use when...» y nombra el caso concreto. Si está vaga, la skill no se activa nunca.

Es una carpeta, no un archivo

Junto al SKILL.md caben plantillas, ejemplos y scripts que la skill usa.

Carga progresiva

Solo el nombre y la descripción están siempre en contexto; el resto entra cuando hace falta.

Un agente: agents/backend-architect.md

```
----  
name: backend-architect  
description: API design, service  
    boundaries, database schemas.  
    Use PROACTIVELY on new services.  
tools: Read, Write, Edit, Bash  
model: opus  
----
```

Eres un arquitecto backend senior.
Empieza por los límites del servicio,
define contratos antes de implementar
y justifica cada decisión de esquema.

Contexto propio

Corre en su ventana aparte: lo que lee no ocupa espacio en tu conversación.

tools limita lo que puede tocar

Un revisor de seguridad lee y no escribe; un implementador sí escribe.

model elige el tier

Haiku para clasificar, Sonnet para implementar, Opus para arquitectura. El coste se reparte.

El cuerpo es su system prompt

No son instrucciones para ti: es la personalidad y el método con los que arranca.

La configuración se extiende a sí misma

Seis skills de la categoría Claude Code Customization escriben las piezas por ti.

<code>create-command</code> Un slash command nuevo en commands/	<code>create-skill</code> Una skill de dominio con su carpeta	<code>create-subagent</code> Un agente especializado propio
<code>create-hooks</code> Ganchos de sesión y de eventos	<code>create-claude-plugin</code> Un plugin empaquetado y distribuible	<code>connect-mcp-server</code> Una conexión a un servidor MCP

CONSECUENCIA Si tu equipo repite un flujo tres veces, ya tiene un comando. Pídeselo.

01

Issue-Driven Development

/issue → /task → /pr → /code-review

De la idea al código revisado



LO QUE GENERA /ISSUE

Un epic padre y sus sub-issues, cada uno con criterios de aceptación, story points, el agente asignado y su grafo de dependencias.

Cuándo usarla

ENCAJA

Features técnicas con requisitos claros

Cuando quieres pasar de la descomposición a la implementación rápido, sin negociar el comportamiento con nadie.

BUSCA OTRA

Si el comportamiento está en discusión

O si tienes un backlog entero esperando en lugar de una sola feature.

Demo: una feature, un PR



```
/issue "Add payment processing"
```

```
/task #101
```

```
/pr
```

Qué mirar: los criterios de aceptación y el agente asignado en el sub-issue.

02

Behavior-Driven Development

`/user-story → /issue → /task`

Un paso más al principio



LO QUE AÑADE

Persona, objetivo y beneficio; escenarios Given/When/Then; y una versión semántica objetivo antes de escribir código.

La especificación en Gherkin

Feature: Autenticación de usuario

Scenario: Login con credenciales válidas

Given un usuario registrado

When envía email y contraseña correctos

Then recibe un token de sesión

And aterriza en su dashboard

Legible por negocio

El stakeholder corrige el escenario sin leer código.

Ejecutable por agentes

Cada escenario se mapea a un criterio de aceptación del sub-issue.

Versionable

La historia fija la versión semántica objetivo desde el inicio.

Las siete fases del workflow completo

01 User story — spec y Gherkin

02 Epic — decomposición y dependencias

03 Tareas — un contexto por sub-issue

04 Implementación — agentes en paralelo

05 Pull request — uno por sub-issue

06 Revisión — agentes especializados

07 Integración y despliegue

EJEMPLO DE REPARTO

backend-architect en la API, frontend-developer en la UI, database-optimizer en el esquema, test-automator en los tests.

Cuándo usarla

ENCAJA

Features de cara al usuario

Cuando el comportamiento tiene que quedar formalizado antes de implementar, o cuando trabajas con stakeholders no técnicos.

BUSCA OTRA

Refactors y trabajo interno

Si no hay un usuario del otro lado, el escenario Gherkin es ceremonia.

Demo: del comportamiento al criterio



```
/user-story
```

```
/issue "Implement user authentication"
```

```
/task #124
```

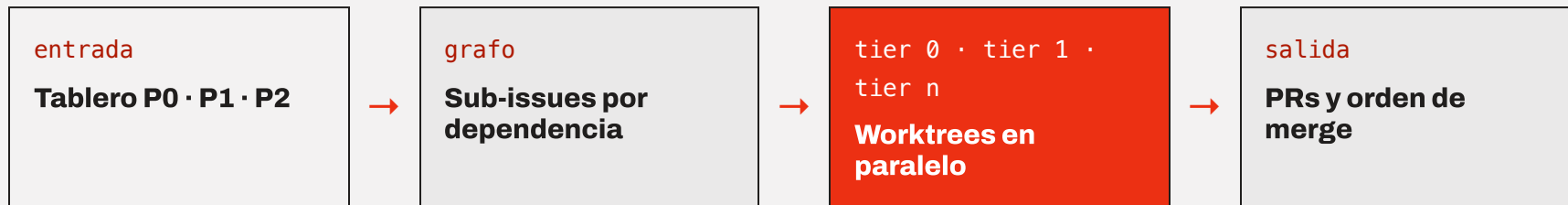
Qué mirar: cómo cada escenario Gherkin se convierte en un criterio de aceptación.

03

Epic-Driven Development

/work-on-opens · agent teams · git worktrees

Del tablero a los worktrees



LO QUE ENVUELVE

Un solo comando ejecuta `/task` y `/pr` por dentro, epic tras epic, en orden de prioridad.

Qué hace `/work-on-opens` con cada epic

`/work-on-opens <project-board-url>`

01 Clarifica integraciones de terceros

02 Construye el grafo de dependencias

03 Agrupa en tiers paralelizables

04 Crea un git worktree por sub-issue

05 Corre `/task` en paralelo, en background

06 Crea los PRs vía `/pr` al completarse

07 Reporta el orden de merge recomendado

Cuándo usarla

ENCAJA

Un backlog entero, un sprint entero

Cuando varios epics necesitan resolverse con el máximo paralelismo posible.

REQUISITOS

Tablero de prioridades mantenido

Más el flag
CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS activo.

Demo: un tablero en paralelo



```
/work-on-opens <board-url>
```

```
git worktree list
```

```
/merge-and-test #123
```

Qué mirar: un worktree por sub-issue y el orden de merge que reporta al final.

Cómo elegir: mira tu entrada

Tengo una feature y los requisitos están claros

Issue-Driven

Tengo un comportamiento que hay que acordar antes

BDD

Tengo un tablero entero esperando

Epic-Driven

PROGRESIÓN

Cada una contiene a la anterior. Aprender la primera es el prerequisite de las otras dos.

Cuatro patrones de orquestación

Secuencial	Los agentes se pasan el contexto en orden	backend-architect → frontend-developer → test-automator
Paralelo	Varios agentes trabajan a la vez y se fusiona el resultado	performance-engineer + database-optimizer
Validación	El trabajo principal pasa por una revisión especializada	payment-integration → security-auditor
Condicional	El agente se elige según el análisis previo	debugger → [backend-architect frontend-developer]

Antes de la primera línea de código

Dos comandos preparan el terreno para cualquiera de las tres metodologías.

`/architecture`

Stack, dominios y conexiones

Define la tecnología, la separación de dominios y los patrones de interconexión antes de implementar.

`/mvp-requirements`

Alcance del MVP

Explora capacidades técnicas a través de la documentación y fija qué entra y qué no en el MVP.

Cheatsheet de comandos

METODOLOGÍAS

/issue "..."	Epic + sub-issues
/user-story	Spec BDD en Gherkin
/work-on-opens	Tablero completo

PLANIFICACIÓN

/architecture	Stack y dominios
/mvp-requirements	Alcance del MVP

EJECUCIÓN

/task #123	Orquesta agentes
/pr	PR con convenciones
/code-review	Revisión especializada
/merge-and-test	Plan de merge + tests
/todos	Seguimiento de tareas

INVESTIGACIÓN

/nlm-research	Research con NotebookLM
/prompt	Asistente de prompts

Próximos pasos

- 01 Clona la configuración en tu ~/ .claude
- 02 Ejecuta /issue en una feature pequeña y real
- 03 Añade /user-story cuando entre negocio
- 04 Deja Epic-Driven para cuando el flujo ya sea rutina

INSTALACIÓN

```
cd ~/.claude
```

```
git clone --recurse-submodules  
git@github.com:ronnycoding/.claude.git .
```

Documentación por carpeta

```
commands/README.md · skills/README.md ·  
agents/README.md
```


Por dónde empezar

Una feature, un comportamiento o un tablero: elige por tu entrada.